instead of one-to-one). These operations could be implemented as a higher-level application using ASK as a blockchain interfacing layer.

FlightID	Airline	FromCity	ToCity	DepTime	SeatsAvail
1	AirlineA	BUF	NYC	6.00 AM	8
2	AirlineA	BUF	NYC	10.00 AM	6
3	AirlineA	BUF	NYC	6.00 PM	7
4	AirlineB	BUF	NYC	7.00 AM	10
5	AirlineB	BUF	NYC	1.00 PM	4
6	AirlineB	BUF	NYC	5.00 AM	2

Figure 6.12 ASK available seats display

Now that you know what information is displayed, let's explore a simple scenario to demonstrate the typical operation of the ASK Dapp. Suppose that a customer who holds a seat on a flight operated by AirlineA finds and wants to change to a seat on a flight operated by AirlineB leaving at 1 p.m. (`flightID` 5 on the ASK display shown in figure 6.12). The customer makes a request to AirlineA to change their seat. An agent at AirlineA issues a request (ASK`Request()`) to AirlineB on the customer's behalf to confirm that the seat is available. An agent at AirlineB examines its system and responds (ASK`Response()`). Assuming that the response is a success response (the seat is available), AirlineA initiates payment to AirlineB, and the ASK display is updated to show the number of seats now left on the flight with `flightID` 5. Other updates related to the seat vacated by this customer on the original flight also happen offline, but they are not shown here. Either of the airlines involved may message the customer about the change, and all the ensuing operations are carried out offline. Most of the operations are offline except for